1. Tangkapan Layar

Membuat Cluster

Mematikan Swap

```
ubuntu@ubuntu:~$ sudo swapoff -a
sudo sed -i '$ s/^\//# \//' /etc/fstab
[sudo] password for ubuntu:
```

Konfigurasi kernel

```
ubuntu@ubuntu:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

sudo sysctl --system
overlay
br_netfilter
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /etc/sysctl.d/10-bufferbloat.conf ...
* Applying /etc/sysctl.d/10-console-messages.conf ...
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-map-count.conf ...
* Applying /etc/sysctl.d/10-network-security.conf ...
* Applying /etc/sysctl.d/10-ptrace.conf ...
* Applying /etc/sysctl.d/10-zeropage.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /usr/lib/sysctl.d/99-protect-links.conf ...
* Applying /etc/sysctl.d/99-sysctl.conf ...
* Applying /etc/sysctl.d/k8s.conf ...
* Applying /etc/sysctl.conf ...
kernel.apparmor_restrict_unprivileged_userns = 1
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
kernel.kptr_restrict = 1
kernel.sysrq = 176
vm.max_map_count = 1048576
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
kernel.yama.ptrace_scope = 1
vm.mmap_min_addr = 65536
kernel.pid_max = 4194304
fs.protected_fifos = 1
fs.protected_hardlinks = 1
fs.protected_regular = 2
fs.protected_symlinks = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

Fathurrahman Kesuma Ridwan 2306206282

Install Containerd

```
ubuntu@ubuntu:~$ sudo apt-get update
sudo apt-get install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml >/dev/null
sudo sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g' /etc/containerd/config.toml
sudo systemctl restart containerd
Hit:1 http://id.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://id.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://id.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  runc
The following NEW packages will be installed:
  containerd runc
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 47.2 MB of archives.
After this operation, 178 MB of additional disk space will be used.
Get:1 http://id.archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.3.3-0ubuntu1~24.04.3 [8,815 kB
]
Get:2 http://id.archive.ubuntu.com/ubuntu noble-updates/main amd64 containerd amd64 1.7.28-0ubuntu1~24.04.2 [3
8.4 MB]
Fetched 47.2 MB in 9s (5,002 kB/s)
Selecting previously unselected package runc.
```

Install kubeadm, kubelet dan kubectl

```
ubuntu@ubuntu:~$ sudo apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/k
ubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.31/deb
/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo apt-get install -y conntrack
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.8).
curl set to manually installed.
gpg is already the newest version (2.4.4-2ubuntu17.4).
gpg set to manually installed.
The following NEW packages will be installed:
  apt-transport-https
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
```

Inisiasi Cluster

```
ubuntu@control-plane:~$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --apiserver-advertise-address=192.168.59.101
I0320 07:09:13.520959    4306 version.go:261] remote version is much newer: v1.35.3; falling back to: stable-1.31
[init] Using Kubernetes version: v1.31.14
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action beforehand using 'kubeadm config images pull'
W0320 07:09:14.257366    4306 checks.go:843] detected that the sandbox image "registry.k8s.io/pause:3.8" of the container runtime is inconsistent with that
used by kubeadm.It is recommended to use "registry.k8s.io/pause:3.10" as the CRI sandbox image.
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [control-plane kubernetes kubernetes.default kubernetes.default.svc kubernetes.default.svc.cluster.lo
cal] and IPs [10.96.0.1 192.168.59.101]
[certs] Generating "apiserver-kubelet-client" certificate and key
[certs] Generating "front-proxy-ca" certificate and key
[certs] Generating "front-proxy-client" certificate and key
[certs] Generating "etcd/ca" certificate and key
[certs] Generating "etcd/server" certificate and key
[certs] etcd/server serving cert is signed for DNS names [control-plane localhost] and IPs [192.168.59.101 127.0.0.1 ::1]
[certs] Generating "etcd/peer" certificate and key
[certs] etcd/peer serving cert is signed for DNS names [control-plane localhost] and IPs [192.168.59.101 127.0.0.1 ::1]
[certs] Generating "etcd/healthcheck-client" certificate and key
[certs] Generating "apiserver-etcd-client" certificate and key
[certs] Generating "sa" key and public key
[kubeconfig] Using kubeconfig folder "/etc/kubernetes"
[kubeconfig] Writing "admin.conf" kubeconfig file
[kubeconfig] Writing "super-admin.conf" kubeconfig file
[kubeconfig] Writing "kubelet.conf" kubeconfig file
[kubeconfig] Writing "controller-manager.conf" kubeconfig file
[kubeconfig] Writing "scheduler.conf" kubeconfig file
[etcd] Creating static Pod manifest for local etcd in "/etc/kubernetes/manifests"
[control-plane] Using manifest folder "/etc/kubernetes/manifests"
[control-plane] Creating static Pod manifest for "kube-apiserver"
[control-plane] Creating static Pod manifest for "kube-controller-manager"
[control-plane] Creating static Pod manifest for "kube-scheduler"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Starting the kubelet
[wait-control-plane] Waiting for the kubelet to boot up the control plane as static Pods from directory "/etc/kubernetes/manifests"
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
```

Command agar kubectl tahu lokasi server kubernetes local pada cluster

```
ubuntu@control-plane:~$ mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Memasukkan worker ke cluster

```
ubuntu@worker-node-1:~$ sudo kubeadm join 192.168.59.101:6443 --token w18wsv.hzoms9w5k4twpcs7 --discovery-token-ca-cert-hash sha256:c15ee24c77348de874e485c7
9300def30b8cadefb298420d6b5e675b4649b918
[sudo] password for ubuntu:
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.597309937s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

```
ubuntu@worker-node-2:~$ sudo kubeadm join 192.168.59.101:6443 --token w18wsv.hzoms9w5k4twpcs7 --discovery-token-ca-cert-hash sha256:c15ee24c77348de874e485c7
9300def30b8cadefb298420d6b5e675b4649b918
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 3.565272424s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

Get nodes

```
ubuntu@control-plane:~$ kubectl get nodes
NAME            STATUS     ROLES           AGE     VERSION
control-plane   NotReady   control-plane   2m32s   v1.31.14
worker-node-1   NotReady   <none>          79s     v1.31.14
worker-node-2   NotReady   <none>          68s     v1.31.14
```

Konfigurasi Jaringan

Install Flannel

```
ubuntu@control-plane:~$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
```

Siapkan Load Balancer

```
ubuntu@control-plane:~$ kubectl edit configmap -n kube-system kube-proxy

[No write since last change]
/bin/bash: line 1: wq: command not found

shell returned 127

Press ENTER or type command to continue
configmap/kube-proxy edited
ubuntu@control-plane:~$ kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.14.8/config/manifests/metallb-native.yam
l
namespace/metallb-system created
customresourcedefinition.apiextensions.k8s.io/bfdprofiles.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bgpadvertisements.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bgppeers.metallb.io created
customresourcedefinition.apiextensions.k8s.io/communities.metallb.io created
customresourcedefinition.apiextensions.k8s.io/ipaddresspools.metallb.io created
customresourcedefinition.apiextensions.k8s.io/l2advertisements.metallb.io created
customresourcedefinition.apiextensions.k8s.io/servicel2statuses.metallb.io created
serviceaccount/controller created
serviceaccount/speaker created
role.rbac.authorization.k8s.io/controller created
role.rbac.authorization.k8s.io/pod-lister created
clusterrole.rbac.authorization.k8s.io/metallb-system:controller created
clusterrole.rbac.authorization.k8s.io/metallb-system:speaker created
rolebinding.rbac.authorization.k8s.io/controller created
rolebinding.rbac.authorization.k8s.io/pod-lister created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:controller created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:speaker created
configmap/metallb-excludel2 created
secret/metallb-webhook-cert created
service/metallb-webhook-service created
deployment.apps/controller created
```

Buat file konfigurasi

```
ubuntu@control-plane:~$ cat <<EOF | kubectl apply -f -
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: first-pool
  namespace: metallb-system
spec:
  addresses:
  - 192.168.59.200-192.168.59.250
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2-adv
  namespace: metallb-system
EOF
ipaddresspool.metallb.io/first-pool created
l2advertisement.metallb.io/l2-adv created
```

Deploy Aplikasi

```
ubuntu@control-plane:~$ kubectl create deployment nginx-test --image=nginx
deployment.apps/nginx-test created
ubuntu@control-plane:~$ kubectl expose deployment nginx-test --port=80 --type=LoadBalancer
service/nginx-test exposed
ubuntu@control-plane:~$ kubectl get pods -o wide
NAME                          READY   STATUS    RESTARTS   AGE     IP           NODE            NOMINATED NODE   READINESS GATES
nginx-test-5b77bfd686-54gnr   1/1     Running   0          4m57s   10.244.2.2   worker-node-2   <none>           <none>
ubuntu@control-plane:~$
```

Rename Cluster untuk kebutuhan soal

```
ubuntu@control-plane:~$ kubectl config rename-context $(kubectl config current-context) k3s-cluster-2306206282
Context "kubernetes-admin@kubernetes" renamed to "k3s-cluster-2306206282".
```

2. Jelaskan apa itu Horizontal Pod Autoscaler dan perannya dalam melakukan scaling pada aplikasi.

Horizontal Pod Autoscaler (HPA) adalah fitur di kubernetes yang secara otomatis menambahkan replika pod berdasarkan metrik tertentu, seperti penggunaan CPU, memori, atau metrik lainnya. HPA menambahkan replika pod ketika metrik melebihi threshold tertentu dan mengurangi replika pod ketika metrik ada di bawah threshold tertentu. Perannya dalam scaling adalah HPA dapat merespons lonjakan *traffic* yang besar secara otomatis tanpa perlu dikonfigurasi lagi secara manual. Pengurangan replika pod secara otomatis juga dapat mengurangi penggunaan

resource dan juga sistem tidak akan mengalami *overload* karena dapat dibatasi dengan maxReplicas dan minReplicas.

3. Misalnya Anda adalah pemilik suatu data center yang memiliki banyak VM dengan beragam spesifikasi.

a. Hal-hal apa saja yang akan Anda jadikan pertimbangan dalam menentukan server mana yang menjadi control plane dan mana yang menjadi worker?

Server yang dipilih untuk control plane adalah server dengan RAM yang besar karena control plane menjalankan komponen-komponen penting, seperti kube-apiserver, scheduler, controller manager, dan ectd sehingga perlu adanya memori yang stabil. Kemudian, perlu CPU yang stabil untuk mengelola state keseluruhan cluster. Selain itu, perlu adanya storage yang cepat karena ectd sensitif terhadap latensi disk. Control plane juga perlu ketersediaan tinggi dan jaringan yang stabil agar komunikasi dengan worker bisa dilakukan dengan cepat.

Server yang dipilih untuk worker node adalah server dengan CPU dan RAM yang besar untuk menjalankan workload aplikasi yang butuh banyak *resourc*e. Kemudian, worker node harus bisa menangani spesifikasi beragam sesuai kebutuhan dan juga harus mudah ditambah dan dihapus untuk *scalability*.

b. Step-step mana saja yang mungkin dapat Anda skip bila membuat cluster melalui cloud provider yang sudah memiliki dedicated service? Berikan penjelasannya.

Service yang menggunakan Managed Kubernetes, seperti Google Kubernetes Engine sudah mengimplementasikan otomatisasi beberapa step, seperti setup control plane, yang termasuk kube-apiserver, scheduler, controller manager, dan etcd. Selain itu, setup etcd cluster, networking cluster, load balancer, dan maintenance control plane juga terotomatisasi. Pembuatan node pool, deploy aplikasi, konfigurasi autoscaling, dan mengurus image container masih perlu dilakukan oleh pengguna.

c. Pada tutorial ini, kita mengekspos aplikasi secara langsung melalui metalLB. Apabila kita meng-handle data sensitif, apa yang Anda akan lakukan untuk menambahkan security layer antara metalLB dan pods?

Saya dapat menambahkan Ingress Controller di antara metalLB dan Pods sehingga routing HTTP, rate limiting, autentikasi, dan TLS termination dapat diimplementasikan. Kemudian, saya akan menggunakan Kubernetes Network Policy untuk membatasi komunikasi antarpods yang dapat membatasi lateral movement ketika terjadinya breach. Selain itu, mTLS dengan Service Mesh juga dapat memastikan komunikasi internal kluster dapat terautentikasi dan terenkripsi. Web Application Firewall juga dapat diimplementasikan untuk menghadapi serangan SQL Injection, XSS dan juga DDOS. Arsitektur daris security layer adalah sebagai berikut:

Client → MetalLB → Ingress+TLS → WAF → Network Policy → Pods

Referensi

https://kubernetes.io/docs/concepts/architecture/
https://docs.cloud.google.com/kubernetes-engine/docs/concepts/kubernetes-engine-overview
https://kubernetes.io/docs/concepts/architecture/